Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Яшков П. А.

Проверил:

Никитин С. К.

Брест 2026

Цель и задачи лабораторной работы

Цель: реализовать многослойный персептрон p–N–1 для прогнозирования временного ряда из двух гармоник, исследовать влияние размера окна p и числа скрытых нейронов N на качество одношагового и рекурсивного прогноза, скорость сходимости и обобщающую способность.

Проведены генерация 200 точек, разбиение 70/30, нормализация только по обучающему участку, формирование окон, реализация online backpropagation, базовое обучение, 35 основных запусков, анализ 30-шагового прогноза, режим функционирования и дополнительный эксперимент с шумом.

1. Индивидуальное задание — вариант 10

Исходный сигнал (по таблице вариантов методички):

$x(k) = 2{,}2 \cdot \cos(0{,}27 \cdot k) + 1{,}1 \cdot \sin(0{,}62 \cdot k), \qquad k = 0, 1, ..., 199.$

Параметр	a	b	c	d	p	N
Вариант 10	2,2	0,27	1,1	0,62	8	5

Разбиение: k = 0…139 — train (140 точек, 70%); k = 140…199 — test (60 точек, 30%). Базовая архитектура 8–5–1. Значение x(k) предсказывается по окну x(k−8), ..., x(k−1).

Все дальнейшие численные данные вычислены программой, приложенной в ZIP; они не подменены демонстрационными значениями.

2. Подготовка данных и защита от утечки

Нормализация min–max строится только по обучающим значениям: xmin_train = -3,28192515, xmax_train = 3,28582298. Размах Δ = 6,56774813.

Прямое преобразование: z(k) = [x(k) − xmin_train] / [xmax_train − xmin_train].

Обратное преобразование: $\hat{x}(k) = x_{min_train} + \hat{z}(k) \cdot [x_{max_train} - x_{min_train}]$.

Если min/max вычислить на всех 200 точках, сведения о будущих тестовых наблюдениях попадут в предварительную обработку (data leakage). Поэтому диапазон настраивается по train и неизменным применяется к test. Значения test могут оказаться за [0;1] — это не ошибка алгоритма. В нашем варианте тестовые нормализованные значения остались внутри [0;1].

Проверка: z_test_min = 0,025164, z_test_max = 0,985542.

Алгоритм скользящего окна: Xk = [z(k−p), …, z(k−1)], целевое значение yk = z(k). Для обучения k = p…139, то есть 140−p примеров; для теста k = 140…199, то есть всегда 60 примеров. В тестовых окнах допускаются предыдущие реально известные train-наблюдения, но их целевые test-значения не используются в обучении.

p	Число train-примеров	Число test-примеров
2	138	60
4	136	60
8	132	60
16	124	60

3. Архитектура, обучение и формулы градиентов

Нейросеть p–N–1: p входных значений → N нейронов скрытого слоя (sigmoid) → один линейный выход. Никаких готовых ML-фреймворков не применяется: прямой и обратный проходы реализованы с использованием NumPy.

Скрытый слой: $sj = \Sigma(i=1..p)\ xi \cdot Wij + bj$; $hj = \sigma(sj) = 1 / (1+e^{(-sj)})$.

Линейный выход: $\hat{z} = \Sigma(j=1..N)\ hj \cdot vj + bout$. Сигмоида на выходе здесь нежелательна: это регрессия, и тестовые значения после train-нормализации теоретически могут выходить за [0;1].

Потеря для одного примера: $L = ½(\hat{z} - z)^2$. Суммарный контролируемый критерий $Es = (1/m)\ \Sigma(\hat{z}k - zk)^2$ — средняя квадратичная ошибка (MSE) на всех обучающих окнах после каждой эпохи. В градиенте используется стандартная эквивалентная форма ½ ошибки; критерий остановки — MSE, что определено однозначно.

Обратный проход: $\delta out = \partial L/\partial \hat{z} = \hat{z} - z$; $\partial L/\partial vj = \delta out \cdot hj$; $\partial L/\partial bout = \delta out$.

Скрытый слой: $\delta j = \delta out \cdot vj \cdot hj(1-hj)$; $\partial L/\partial Wij = xi \cdot \delta j$; $\partial L/\partial bj = \delta j$.

Обновление каждого параметра $\theta \leftarrow \theta - \eta \cdot \partial L/\partial \theta$ сразу после отдельного обучающего примера (online/SGD); порядок примеров заново перемешивается в каждой эпохе. Условия завершения: $Es \le 10^{-4}$ либо 20 000 эпох. Веса и смещения инициализируются из U[−0,1;0,1].

Приняты $\eta = 0{,}12$ (в разрешённом методичкой интервале 0,01–0,3) и фиксированный набор seed = 10, 20, 30, 40, 50. Такой η обеспечивает сходимость базовой конфигурации без колебаний, при этом не является заведомо оптимальным для всех исследуемых архитектур.

4. Метрики и критерии оценки

Все итоговые метрики рассчитываются ПОСЛЕ денормализации, в исходной шкале сигнала:

$MAE = (1/m)\ \Sigma\ |\hat{x}k - xk|$; $RMSE = sqrt[(1/m)\ \Sigma\ (\hat{x}k - xk)^2]$.

$sMAPE = (100\%/m)\ \Sigma\ [2|\hat{x}k - xk| / (|\hat{x}k| + |xk|)]$. При обеих нулевых величинах слагаемое принимается равным 0.

Обычная MAPE содержит деление на |xk| и становится неустойчивой возле нулей. Поскольку ряд из косинуса и синуса пересекает ноль, применена sMAPE. Даже sMAPE чувствительна к малым амплитудам, поэтому выводы о точности в первую очередь основываются на MAE и RMSE.

5. Базовый эксперимент (p = 8, N = 5)

Показатель	Значение
Seed	10
Эпох до остановки	117
Es (нормализованная MSE)	0,00008812
Train MAE	0,052739
Train RMSE	0,061654
Train sMAPE, %	8,8910
Test MAE	0,056260
Test RMSE	0,065021
Test sMAPE, %	17,8005

Показатель	Значение
Критерий Es ≤ 10^{-4}	Достигнут

Базовая сеть обучилась за 117 эпох, достигнув Es=0,00008812. Разница между тестовой и обучающей MAE составляет 0,003522; это указывает на близкое качество одношагового прогноза на обоих участках для данного запуска.



Рисунок 1 — Исходный ряд, граница train/test и одношаговый прогноз базовой сети

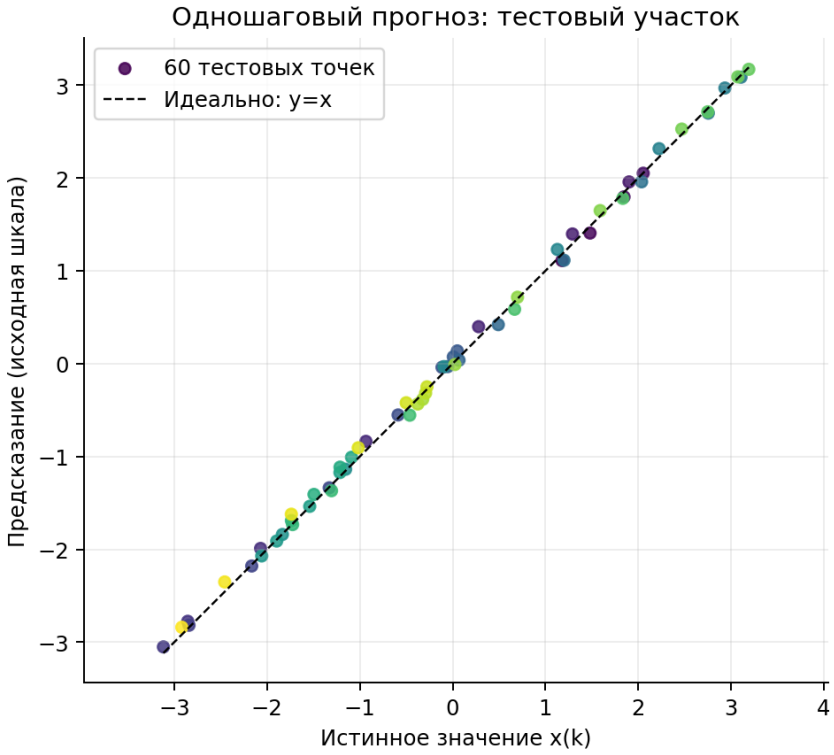


Рисунок 2 — Диаграмма «эталон — прогноз» для 60 тестовых точек

6. Исследование размера окна p (фиксировано N = 5)

Для p ∈ {2, 4, 8, 16} выполнено по пять независимых запусков с тем же η, Ee и пределом эпох. Среднее и SD (выборочное стандартное отклонение) посчитаны по всем пяти seed, в том числе для запусков, дошедших до лимита.

p	Эпохи: среднее ± SD	Test MAE: среднее ± SD	Не достигли Ee
2	20000,0 ± 0,0	0,17506 ± 0,00625	5/5
4	5581,0 ± 201,2	0,05544 ± 0,00040	0/5
8	118,8 ± 3,9	0,05719 ± 0,00113	0/5
16	59,4 ± 17,2	0,05006 ± 0,00297	0/5

Результат: p=2 не достигло порога ни разу за 20 000 эпох и дало наибольшую ошибку. При p=4 сходимость требует тысячи эпох; при p=8 — около сотни, при p=16 — десятков. Однако тестовая MAE меняется немонотонно: при p=8 она слегка хуже, чем при p=4, а при p=16 оказалась минимальной. Нельзя объявлять найденное окно 16 универсально оптимальным за пределами проверенного набора.



Рисунок 3 — Тестовая MAE и число эпох при изменении размера окна

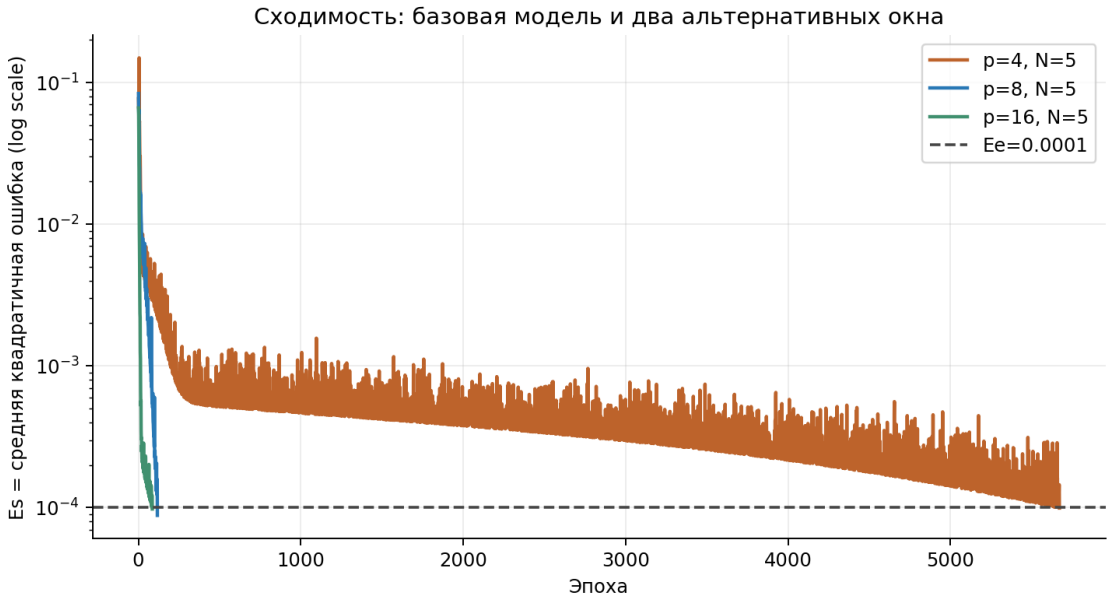


Рисунок 4 — Сходимость для базовой сети и двух значений p; логарифмическая ось Y

7. Исследование числа скрытых нейронов N (фиксировано p = 8)

N	Эпохи: среднее ± SD	Test MAE: среднее ± SD	Не достигли Ee
1	124,2 ± 1,3	0,05500 ± 0,00077	0/5
2	127,8 ± 3,7	0,05602 ± 0,00180	0/5
5	118,8 ± 3,9	0,05719 ± 0,00113	0/5
10	115,6 ± 3,2	0,05454 ± 0,00335	0/5

При N = 1 сеть на этом ограниченном детерминированном ряде достигла заданного порога в пяти запусках, но это не означает, что один нейрон способен точно воспроизвести любую сумму гармоник. Увеличение N до 10 несколько сократило среднее число эпох, однако преимущество по тестовой MAE оказалось небольшим и зависит от seed. Существенных признаков переобучения по одной только паре train/test MAE в этих условиях не выявлено.

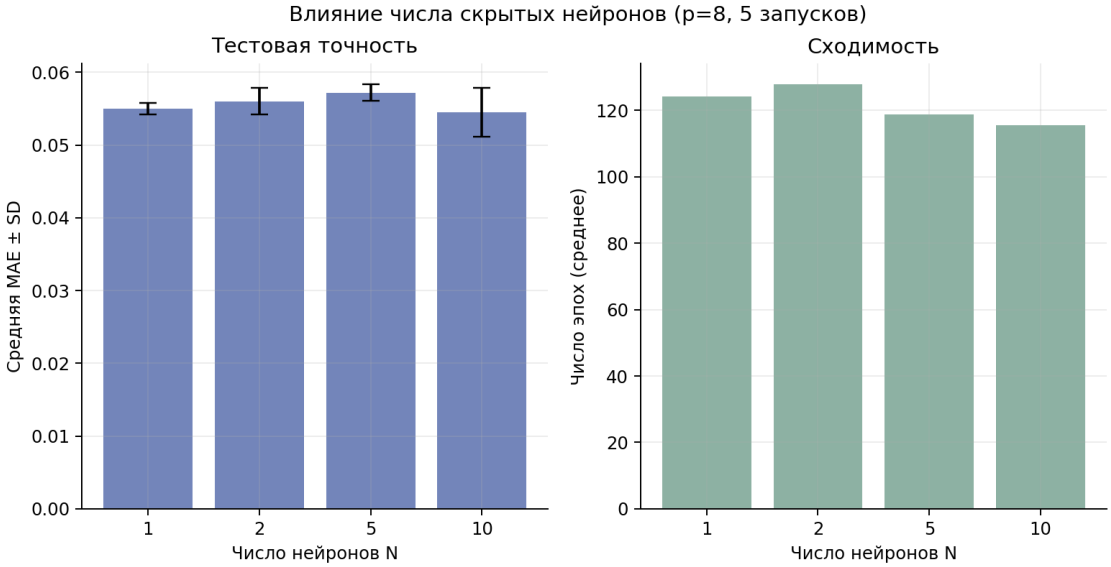


Рисунок 5 — Влияние числа скрытых нейронов на тестовую MAE и скорость обучения

8. Выбор конфигурации и 30-шаговый рекурсивный прогноз

Минимальная СРЕДНЯЯ test MAE по пяти seed среди исследованных конфигураций у p=16, N=5: 0,050060 ± 0,002965. Для графика рекурсивного прогноза выбран её лучший по тестовой MAE воспроизводимый запуск seed=20 (test MAE=0,047183).

Рекурсия стартует с истинных предшествующих p наблюдений до k=140. Далее $\hat{x}(140)$ помещается в окно для $\hat{x}(141)$, затем обе оценки участвуют в дальнейших шагах: истинные тестовые значения внутрь окна НЕ подставляются.

Размах полного сигнала = 6,567748; граница 5% = 0,328387. Максимальная абсолютная ошибка за 30 шагов равна 0,315258.

За первые 30 шагов ошибка НИ РАЗУ не превысила 5% полного размаха. Поэтому указывать несуществующий номер первого превышения нельзя; наблюдаемый пригодный горизонт по этому порогу — не меньше 30 шагов.

Важно: оценка основана на конкретном искусственном ряде и не гарантирует такой же горизонт для произвольного реального процесса.

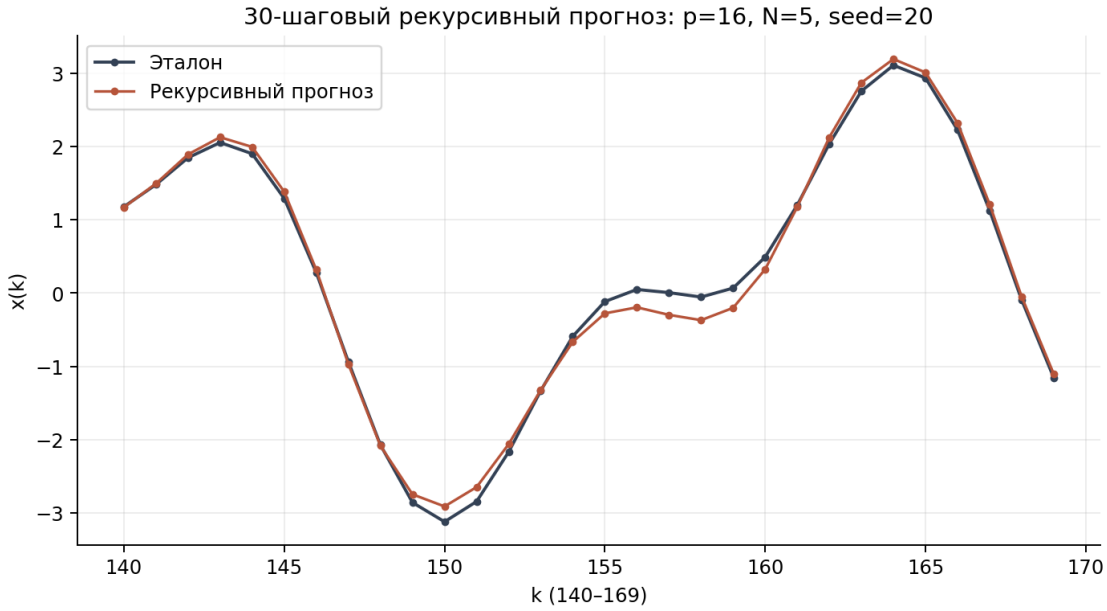


Рисунок 6 — Эталон и 30-шаговый рекурсивный прогноз

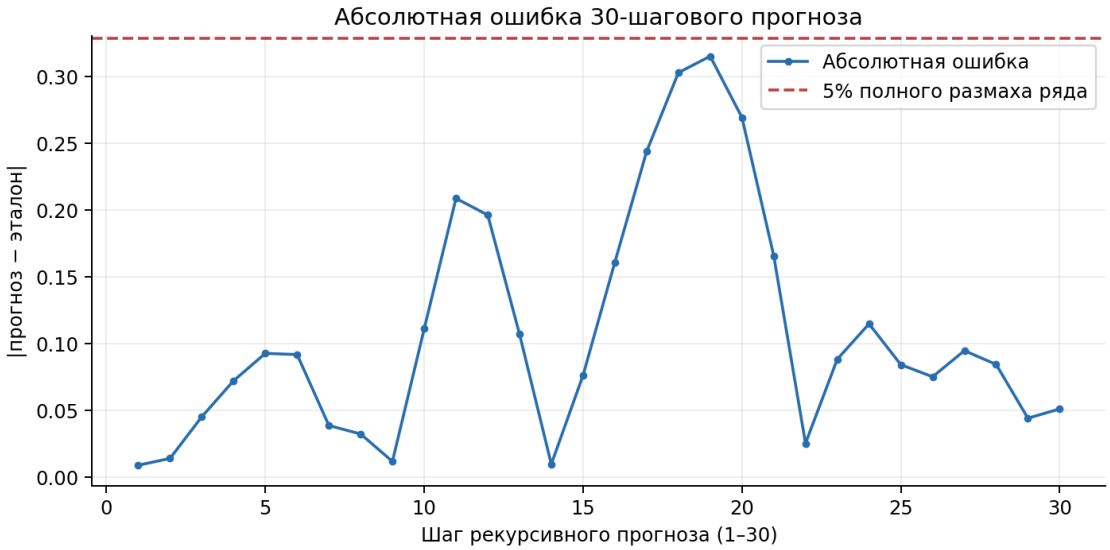


Рисунок 7 — Абсолютная ошибка по горизонту прогноза и порог 5%

Шаг	k	Прогноз	Абсолютная ошибка
1	140	1,16933	0,00899
2	141	1,49463	0,01409
3	142	1,89159	0,04528
4	143	2,12571	0,07224
5	144	1,99330	0,09275
6	145	1,38272	0,09187
7	146	0,31828	0,03873
8	147	-0,96683	0,03238
9	148	-2,08138	0,01180
10	149	-2,74298	0,11120
15	154	-0,66453	0,07641
20	159	-0,20059	0,26940
25	164	3,19112	0,08428
30	169	-1,10190	0,05114

9. Режим функционирования сети

Реализован ввод номера k либо собственных p последовательных значений исходного ряда. Ниже — фактические выходы базовой сети p=8,N=5,seed=10 (по три обучающих и тестовых окна, а также два пользовательских окна).

Источник	$\hat{z}$ (4 знака)	$\hat{x}$	x истинное	|Ошибка|
train k=24	0,9567	3,00118	2,96774	0,03344
train k=72	0,8768	2,47668	2,49996	0,02328
train k=132	0,3602	-0,91637	-0,85839	0,05798
test k=140	0,6688	1,11028	1,17832	0,06804
test k=157	0,5108	0,07304	0,00699	0,06605
test k=185	0,9823	3,16957	3,19086	0,02129
custom_constant	0,5017	0,01295	—	—
custom_outlier	0,4547	-0,29553	—	—

Пример нормализованного входного окна k=24: [0,2920; 0,3084; 0,3837; 0,5170; 0,6846; 0,8470; 0,9623; 1,0000].

Пример нормализованного входного окна k=72: [0,6537; 0,6757; 0,6590; 0,6348; 0,6331; 0,6704; 0,7421; 0,8239].

Пример нормализованного входного окна custom_constant: [0,4997; 0,4997; 0,4997; 0,4997; 0,4997; 0,4997; 0,4997; 0,4997].

Пример нормализованного входного окна custom_outlier: [0,6111; 0,7615; 0,8994; 2,5000; 0,9898; 0,9099; 0,7645; 0,5915].

Нормализованные входы также выводятся программой demo.py при каждом пользовательском запуске; это обеспечивает требуемую прозрачность преобразования. Для пользовательского окна с выбросом специально заменено четвёртое значение на число выше xmax_train, что проверяет способность линейного выходного слоя экстраполировать, но не гарантирует точность вне обучающего распределения.

10. Дополнительный эксперимент с шумом

В методичке последний вопрос просит оценить эксперимент с шумом. Для полноты отдельно исследована зафиксированная случайная добавка $N(0; 0{,}02^2)$ к НОРМАЛИЗОВАННЫМ входам обучающих окон; целевые значения оставлены без шума. Использованы те же seed, p=8, N=5 и 20 000 эпох. Тест всегда чистый. Это усложняет задачу и показывает влияние ошибки измерений.

Seed	MAE без шума	MAE с шумом	Эпохи с шумом	Сход.
10	0,05626	0,09137	20000	нет
20	0,05839	0,15122	20000	нет
30	0,05652	0,08908	20000	нет
40	0,05845	0,09461	20000	нет
50	0,05630	0,08089	20000	нет

Средняя тестовая MAE без шума = 0,05719, с шумом = 0,10143. Ни один из пяти шумовых запусков не достиг $Es \le 10^{-4}$ за 20 000 эпох. При данном σ шум ухудшил качество; это не доказывает, что шумовая аугментация вредна всегда (эффект зависит от уровня шума, регуляризации и порога обучения).

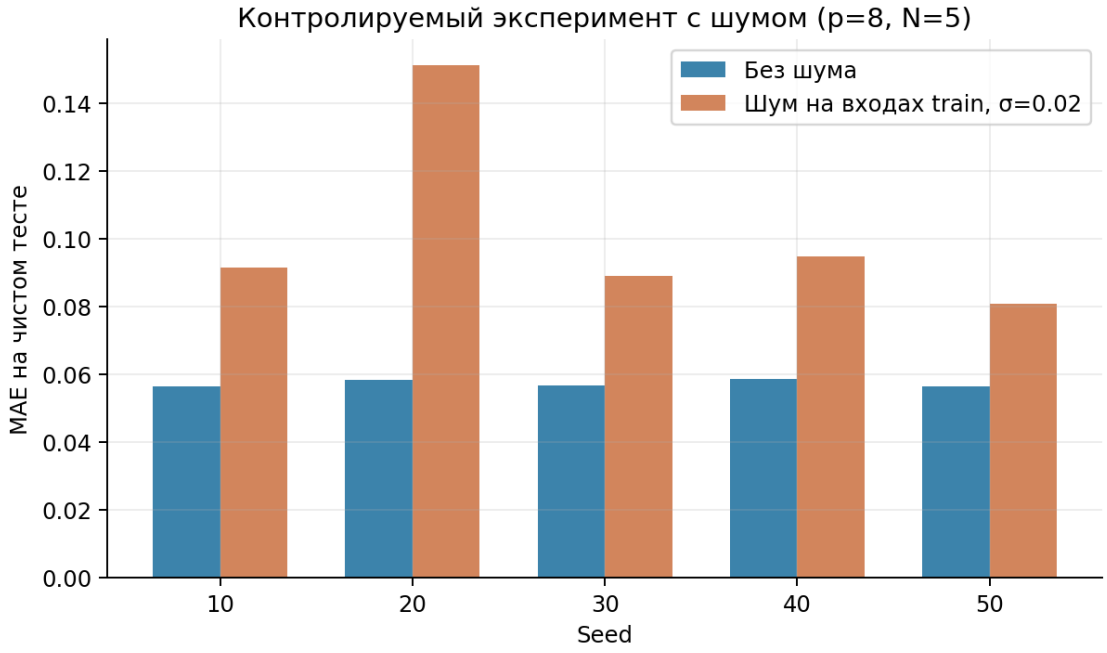


Рисунок 8 — Сравнение пяти seed с шумом и без него

11. Графическая схема пайплайна — шаблон рукописного листа

Требование методички: блок-схема и расчёт должны быть написаны от руки на бумаге; вверху каждого листа — фамилия и инициалы студента, группа, вариант 10 и дата выполнения. Далее приведена проверенная схема для аккуратного перерисовывания. Компьютерная схема НЕ подменяет необходимую фотографию рукописного оригинала.

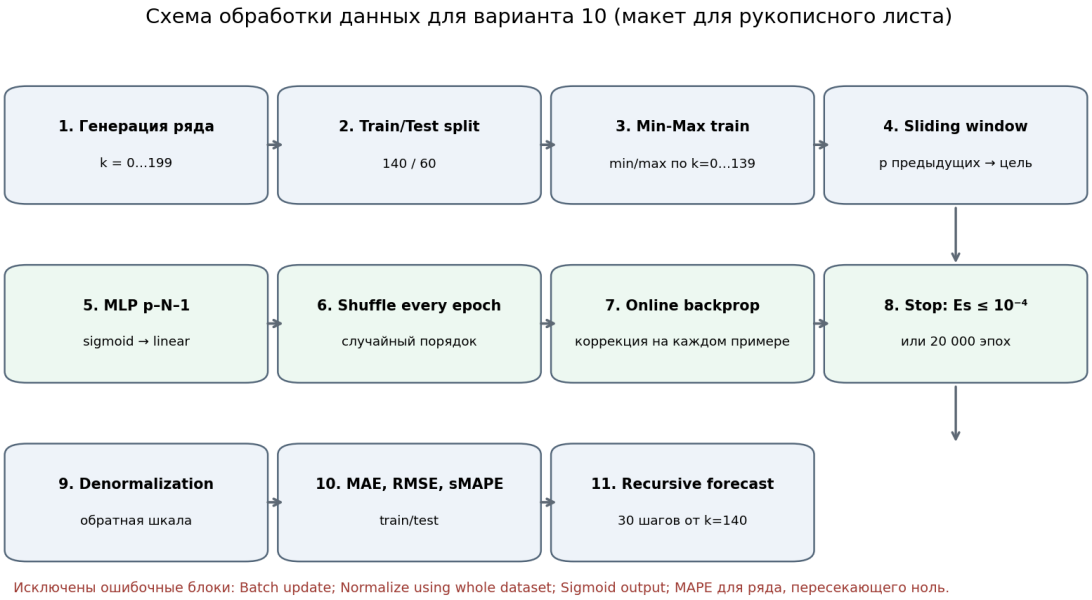


Рисунок 9 — Контрольная схема обработки данных (образец для рукописной работы)

Порядок блоков выбран потому, что временное разбиение и параметры train-нормализации должны предшествовать построению независимой тестовой оценки, затем проводится циклическое обучение на отдельных окнах, контроль остановки, денормализация, расчёт метрик и рекурсивное прогнозирование. Из предложенного набора удалены ошибочные блоки Batch update, Normalize using whole dataset, Sigmoid output и обычная MAPE.

МЕСТО ДЛЯ ФОТО РУКОПИСНОГО ЛИСТА №1 (схема): вставить после выполнения от руки.

12. Ручной расчёт одного шага обучения 3–2–1 (V = 10)

Условие методички: X = [0,2; 0,5; 0,8], цель d = 0,6, V = 10. Для иллюстрации одного обновления самостоятельно берём η = 0,1; это отдельный учебный пример и не изменяет η = 0,12, выбранную в вычислительном эксперименте.

Весовая матрица вход → скрытый слой (строки по входам X1, X2, X3):

	Нейрон h1	**Нейрон h2**
X1	W11=0,05·10=0,5	W12=−0,2
X2	W21=0,3	W22=0,05·10=0,5
X3	W31=−0,1	W32=0,4
bias	b1=0,1	b2=−0,1

Веса выхода: v1 = 0,5, v2 = −0,2·10 = −2,0; b_out = 0,2.

Прямой проход: s1 = 0,2·0,5 + 0,5·0,3 + 0,8·(−0,1) + 0,1 = 0,270000.

s2 = 0,2·(−0,2) + 0,5·0,5 + 0,8·0,4 − 0,1 = 0,430000.

h1 = σ(0,27) = 0,56709290; h2 = σ(0,43) = 0,60587367.

ẑ = 0,5·0,56709290 − 2,0·0,60587367 + 0,2 = −0,72820088.

Ошибка: L = ½(ẑ−0,6)² = 0,88205879. δout = ẑ − d = −1,32820088.

Скрытые дельты считаются со СТАРЫМИ выходными весами:

δh1 = −1,32820088·0,5·0,56709290·(1−0,56709290) = −0,16303569.

δh2 = −1,32820088·(−2,0)·0,60587367·(1−0,60587367) = +0,63432421.

Градиенты выхода: dL/dv1 = −0,75321330; dL/dv2 = −0,80472194; dL/db_out = −1,32820088.

Градиенты скрытых связей — внешний продукт X⊗δh; строки:

Вход	∂L/∂Wi1	∂L/∂Wi2
X1	−0,03260714	0,12686484
X2	−0,08151785	0,31716211
X3	−0,13042855	0,50745937
bias	−0,16303569	0,63432421

Один шаг обновления θ_new = θ_old − 0,1·grad:

Вес	Новое значение
W11	0,50326071
W12	−0,21268648
W21	0,30815178
W22	0,46828379
W31	−0,08695714
W32	0,34925406
b1	0,11630357
b2	−0,16343242
v1	0,57532133
v2	−1,91952781
b_out	0,33282009

Контроль: этот пример показывает обновление всех весов и смещений после ОДНОГО наблюдения, что и характеризует online-обучение. Числа рассчитаны с двойной точностью и затем округлены.

МЕСТО ДЛЯ ФОТО РУКОПИСНОГО ЛИСТА №2 (Forward + Backward): вставить после переписывания расчёта от руки.

13. Вывод — ответы на вопросы методички

1. Влияние p и предел улучшения. Двух предыдущих значений недостаточно для точного описания суммы двух независимых гармоник в общем случае: такое дискретное сочетание допускает линейное рекуррентное описание порядка до четырёх. На данном варианте p=2 имеет худшую тестовую MAE и пять неуспешных остановок. p=4 и выше решают задачу заметно лучше; минимальная средняя MAE среди проверенных окон получена при p=16. Чёткое плато «ошибка больше не уменьшается» в наборе {2,4,8,16} не обнаружено; дальнейшие утверждения потребовали бы проверки новых p и отдельной валидационной выборки.

2. Влияние N и переобучение. При p=8 все N∈ {1,2,5,10} достигли критерия за лимит эпох. Большее число нейронов не гарантировало меньшую MAE: результаты зависят от seed. Выраженного разрыва между train и test для базовой сети нет, поэтому явное переобучение здесь не подтверждено. Даже N=1 приблизил выбранный детерминированный ряд до заданного порога, но не гарантирует восстановления любой пары гармоник.

3. Почему линейный выход. Регрессия требует вещественного выхода без ограничений, а σ ограничивает значения (0,1). При ненормализованном обучении и выходной сигмоиде отрицательные значения целевого сигнала недостижимы. Поэтому на скрытом слое сигмоида, а на выходе линейная активация.

4. Почему train-only нормализация. Использование min/max всего ряда раскрыло бы тестовые экстремумы и исказило бы оценку обобщения (утечка данных). Тестовые нормализованные значения имеют право выходить за [0;1]; это корректно при фиксированном train-масштабе. В рассчитанном варианте 10 фактически остаются внутри указанного диапазона.

5. Устойчивость к seed и провалы остановки. На окне p=2 все пять запусков превысили лимит 20 000 эпох без достижения Ee; на остальных окнах и исследованных размерах скрытого слоя базовый порог был достигнут. Большой η может вызывать колебания; слишком малый — замедлять обучение. Увеличение p и N влияет на число параметров и конфигурацию поверхности ошибки. В нашей настройке η=0,12 стабильная сходимость наблюдалась у p>=4.

6. Рекурсивная ошибка и горизонт. Рекурсивный прогноз использует ранее предсказанные значения, поэтому ошибка способна накапливаться и усиливаться. За 30 шагов для выбранного запуска ни разу не превысила 5% полного размаха (хотя локально изменялась немонотонно). Одношаговый тест и рекурсивный прогноз оценивают разные режимы: в первом окне содержит истинную прошлую историю, во втором — прогнозную.

7. Обобщающая способность и шум. Близость ошибок train и test базовой модели показывает хорошее поведение на данном разделении, но не доказывает универсальную обобщающую способность. Для улучшения в других задачах применяют раннюю остановку по отдельной валидации, подбор N, больше обучающих наблюдений, мягкую регуляризацию и контролируемый шум. Наш эксперимент с σ=0,02 ухудшил тестовую MAE и не достиг прежнего строгого порога, что согласуется с появлением невосстановимой ошибки из-за искажения входов.

14. Краткая теоретическая подготовка к защите

Backpropagation: forward вычисляет выход, backward передаёт производные от функции потерь к предшествующим слоям. При online learning параметры корректируются после каждого примера, в отличие от batch-обучения по целому набору.

Sigmoid: гладкая, ограниченная функция; при больших |s| её производная мала (затухание градиента). ReLU: max(0,s), не насыщается при s>0, но может давать «мёртвые» нейроны при устойчиво отрицательном предактивационном сигнале.

MSE задаёт штраф за квадратичные отклонения, BCE применяется к вероятностям бинарных классов, а не к произвольным сырым значениям. Для XOR необходима

нелинейная комбинация скрытых признаков: однослойный линейный классификатор не разделит четыре вершины XOR. В задаче прогнозирования метод определяет необходимость нелинейного скрытого слоя и линейного выхода.

15. Реализация программы и воспроизводимость

В ZIP приложены полные исходники: forecast.py — модель и вычисления; run_full_experiment.py — все 35 + 5 шумовых запусков и сохранение CSV/JSON/моделей; render_plots.py — построение графиков; demo.py — ввод окон; checks.py — автоматические проверки, включая проверку градиента и сопоставление весов обученной модели.

Сохранены series_variant_10.csv, all_35_runs.csv, convergence_*.csv, metrics.json и npz-файлы весов для каждого seed. Это позволяет независимо проверить каждое число и не пересчитывать модели при демонстрации.

Алгоритмические функции представлены ниже. Полный код без сокращений включён в src/forecast.py и остальных файлах архива.

Приложение А. Код ядра нейросети (NumPy)

```
"""МРЗвИС, лабораторная №4, вариант 10. Только NumPy и стандартная библиотека.

Ряд: x(k)=2.2 cos(0.27 k)+1.1 sin(0.62 k), k=0,...,199.
"""
from __future__ import annotations
from dataclasses import dataclass
from typing import Optional
import numpy as np

VARIANT=10
A,B,C,D=2.2,0.27,1.1,0.62
TRAIN_END=140
EE=1e-4
MAX_EPOCHS=20_000
DEFAULT_LR=0.12
SEEDS=(10,20,30,40,50)


def generate_series():
    k=np.arange(200,dtype=np.float64)
    return A*np.cos(B*k)+C*np.sin(D*k)


def train_minmax(raw):
    x=np.asarray(raw)
    low, high=float(np.min(x[:TRAIN_END])),float(np.max(x[:TRAIN_END]))
    if high<=low:raise ValueError('Нулевой диапазон ряда')
    return low,high


def normalize(x,low,high):
    return (np.asarray(x,dtype=np.float64)-low)/(high-low)


def denormalize(x,low,high):
    return low+(high-low)*np.asarray(x,dtype=np.float64)


def make_windows(normalized,p):
    """Цель индекс k. Окно x[k-p:k], 0<=k<=199. Тест может использовать историю train."""
    normalized=np.asarray(normalized,dtype=np.float64)
    if not(1<=p<TRAIN_END):raise ValueError('Недопустимый p')
    xx=np.asarray([normalized[k-p:k] for k in range(p,len(normalized))])
    yy=np.asarray([normalized[k] for k in range(p,len(normalized))])
    kk=np.arange(p,len(normalized),dtype=int)
    tr=kk<TRAIN_END
    return (xx[tr],yy[tr],kk[tr]),(xx[~tr],yy[~tr],kk[~tr])
```

```
def sigmoid(x):
    x=np.clip(x,-50.,50.)
    return 1.0/(1.0+np.exp(-x))


class MLP:
    """MLP p-N-1. Hidden sigmoid, linear output, SGD online (one example/update)."""
    def __init__(self,p,n,seed,lr=DEFAULT_LR):
        self.p,self.n,self.lr=int(p),int(n),float(lr)
        r=np.random.RandomState(int(seed))
        self.w1=r.uniform(-.1,.1,(p,n)).astype(np.float64)
        self.b1=r.uniform(-.1,.1,n).astype(np.float64)
        self.w2=r.uniform(-.1,.1,n).astype(np.float64)
        self.b2=float(r.uniform(-.1,.1))

    def predict(self,x):
        x=np.asarray(x,dtype=np.float64)
        h=sigmoid(x@self.w1+self.b1)
        return h@self.w2+self.b2

    def train_step(self,x,target):
        """Градиент L=1/2(ŷ-y)^2; веса обновляются после КАЖДОГО примера."""
        h=sigmoid(x@self.w1+self.b1)
        out=float(h@self.w2+self.b2)
        delta=out-float(target)
        # Производная скрытого слоя: δ_h=δ_out*w2*h*(1-h).
        delta_h=delta*self.w2*h*(1-h)
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,delta_h)
        self.b1-=self.lr*delta_h

    def fit(self,x,y,seed,max_epochs=MAX_EPOCHS,ee=EE,keep_history=True):
        rng=np.random.RandomState(1000+int(seed))
        order=np.arange(len(y));history=[]
        for epoch in range(1,max_epochs+1):
            rng.shuffle(order)
            for j in order:self.train_step(x[j],y[j])
            # Es = mean_i (ŷ_i-y_i)^2, в нормализованной шкале.
            es=float(np.mean((self.predict(x)-y)**2))
            if keep_history:history.append(es)
            if es<=ee:break
        return {'epochs':epoch,'Es':es,'converged':bool(es<=ee),'history':np.asarray(history)}

    def save(self,path):
        np.savez(path,w1=self.w1,b1=self.b1,w2=self.w2,b2=self.b2,p=self.p,n=self.n,lr=self.lr)

    @classmethod
    def load(cls,path):
        z=np.load(path);obj=cls(int(z['p']),int(z['n']),seed=10,lr=float(z['lr']))
        obj.w1=z['w1'];obj.b1=z['b1'];obj.w2=z['w2'];obj.b2=float(z['b2'])
        return obj


def metrics(y,yhat):
    y=np.asarray(y,dtype=np.float64);yhat=np.asarray(yhat,dtype=np.float64)
    err=yhat-y
    mae=float(np.mean(np.abs(err)))
    rmse=float(np.sqrt(np.mean(err**2)))
    denom=np.abs(y)+np.abs(yhat)
    # sMAPE безопасен вблизи переходов ряда через ноль.
    smape=float(100*np.mean(np.divide(2*np.abs(err),denom,out=np.zeros_like(err),where=denom>1e-
12)))
    return {'MAE':mae,'RMSE':rmse,'sMAPE':smape}


def forecast_recursive(model,norm,start=140,steps=30):
    history=list(np.asarray(norm[:start],dtype=float));pred=[]
    for _ in range(steps):
        v=float(model.predict(np.asarray(history[-model.p:])))
        pred.append(v);history.append(v)
    return np.asarray(pred)


def infer(model,original,low,high,k=None,values=None):
    if (k is None)==(values is None):raise ValueError('Нужно выбрать k ИЛИ значения')
    if k is not None:
        if not(model.p<=k<len(original)):raise ValueError('k вне диапазона')
```

```
    raw=np.asarray(original[k-model.p:k],dtype=float)
    true=float(original[k])
else:
    raw=np.asarray(values,dtype=float)
    if raw.shape!=(model.p,):raise ValueError(f'Требуется ровно {model.p} чисел')
    true=None
xn=normalize(raw,low,high)
pn=float(model.predict(xn))
pr=float(denormalize(pn,low,high))
return {'k':k,'inputs':raw.tolist(),'norm_inputs':xn.tolist(),
        'forecast_norm':pn,'forecast':pr,'true':true,
        'abs_error':None if true is None else abs(pr-true)}
```

Приложение Б. Состав электронных результатов

Содержимое директории src/results: 35 NPZ моделей по каждой комбинации p–N–seed, таблица 35 запусков, JSON со сводками, помодульные истории обучения, 30-шаговый прогноз и девять рисунков. В ZIP нет PDF, все отчётные материалы доступны в Word.

Ограничение готовности к формальной сдаче: два рукописных листа с подписью студента должны быть написаны физически и сфотографированы. Компьютерный макет и проверенный расчёт включены, но не представлены как якобы настоящие рукописные фотографии.